NLP 220 Section 2

sklearn, NB by hand, practice 3-6

Outline (starting @ 3:35)

Lecture 3

- Probability Review
- Feature Engineering
- Sklearn basics

Lecture 4

- Naive Bayes
- Decision Trees
- SVMs

This Section:

- Probability Review: Practice 3
- Naive Bayes: Practice 5
- Sklearn: Practice 4
- Entropy: Practice 6

Practice Not Graded 3

Probability Review

(a)

A box of 14 candies

8 red and sweet, 2 brown and sour, 4 brown and sweet

- You close your eyes and randomly pick a candy

After you pick the candy, you open your eyes and see it is brown, now what is the probability that it is sweet?

Practice Not Graded 3

Probability Review

(a)

A box of 14 candies

8 red and sweet, 2 brown and sour, 4 brown and sweet

- You close your eyes and randomly pick a candy

After you pick the candy, you open your eyes and see it is brown, now what is the probability that it is sweet?

$$P(S|B) = \frac{P(S \cap B)}{P(B)} = \frac{\frac{|S \cap B|}{|\Omega|}}{\frac{|B|}{|\Omega|}} = \frac{\frac{4}{14}}{\frac{6}{14}} = \frac{4}{6}$$

$$\Omega = (\text{color} \times \text{taste}) = \mathcal{C} \times \mathcal{T}$$

Practice Not Graded 3

Probability Review

(a)

A box of 14 candies

2 brown and sour, 4 brown and sweet

- You close your eyes and randomly pick a candy

After you pick the candy, you open your eyes and see **it is brown,** now what is the probability that it is sweet?

$$P(S|B) = \frac{P(S \cap B)}{P(B)} = \frac{\frac{|S \cap B|}{|\Omega|}}{\frac{|B|}{|\Omega|}} = \frac{\frac{4}{14}}{\frac{6}{14}} = \frac{4}{6}$$

$$P(S) = \frac{|S|}{|\Omega_{brown}|} = \frac{4}{6}$$

$$\Omega = (\text{color} \times \text{taste}) = \mathcal{C} \times \mathcal{T}$$

Practice Not Graded 3

Probability Review

(b)

A box of 10 candies

5 red and sweet, 3 green and sour, 2 green and sweet

You close your eyes and randomly pick a candy, what is the probability that it is sweet?

Practice Not Graded 3

Probability Review

(b)

A box of 10 candies

5 red and sweet, 3 green and sour, 2 green and sweet

You close your eyes and randomly pick a candy, what is the probability that it is sweet?

$$P(S) = \frac{7}{10}$$

$$\Omega = \text{taste} = \mathcal{T} = \{\text{sweet}, \text{sour}\}$$

Practice Not Graded 3

Probability Review

(c)

One marble jar has several different colored marbles inside of it. It has 1 red, 2 green, 4 blue, and 8 yellow marbles. All the marbles are the same size and shape. If Peter takes out a marble from the jar without looking, what is the probability that he will NOT choose a yellow marble.

Practice Not Graded 3

Probability Review

(c)

One marble jar has several different colored marbles inside of it. It has 1 red, 2 green, 4 blue, and 8 yellow marbles. All the marbles are the same size and shape. If Peter takes out a marble from the jar without looking, what is the probability that he will NOT choose a yellow marble.

$$P(\bar{Y}) = 1 - P(Y) = 1 - \frac{8}{15} = \frac{7}{15}$$

$$\Omega = \text{color} = \mathcal{C} = \{R, G, B, Y\}$$

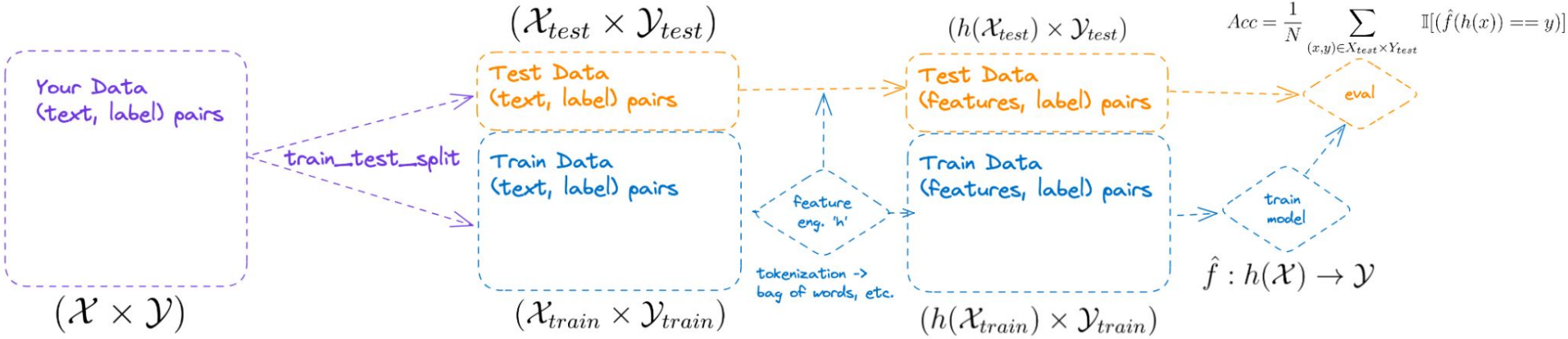
Practice Not Graded 5

Implement a naive bayes classifier using python (without using sklearn):

https://machinelearningmastery.com/naive-bayes-classifier-scratch-python/

Won’t actually do, but will give an overview of an implementation

ML for text-classification overview

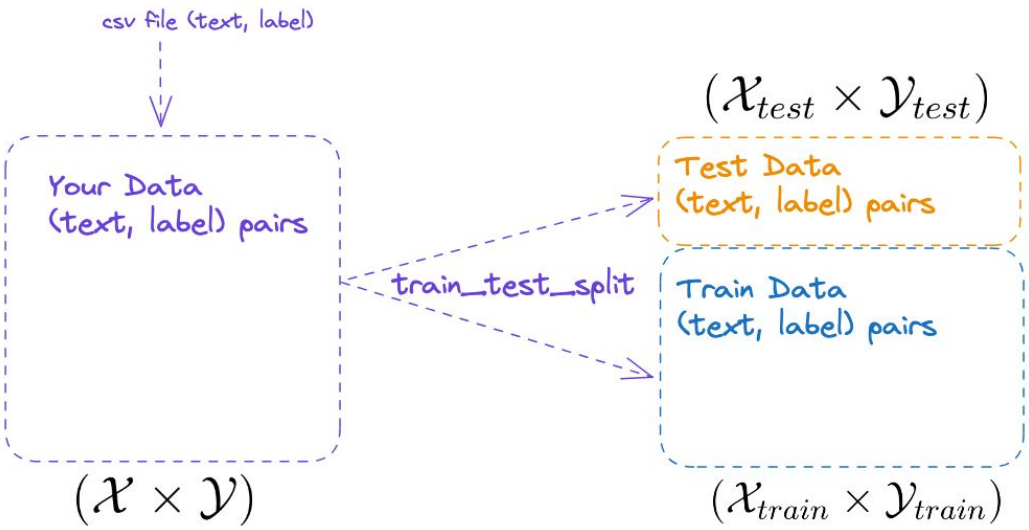


Step 1: read data, split into train/test

Given: csv with text, label header & randomly ordered rows

I use two methods:

```
read_data(path: str) ->

    List[Tuple[str, int]

train_test_split(data: List[Tuple[str,
int]]) -> ...
```

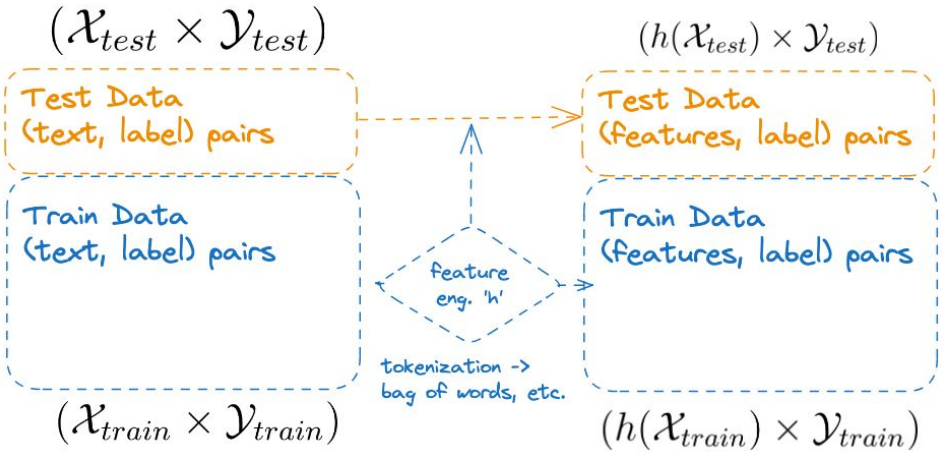


Step 2: Feature engineering

1. Build a vocabulary using only training data (later, ignore unseen tokens or use an UNK token)
2. Use the vocab to turn each sentence into a vector of size |V|

```
build_vocab(train_data) ->

      Dict[str, int]

      # nltk/spacy to tokenize, then counter ->
most_common(vocab_size)
```

```
featurize(data, vocab) -> npt.NDArray:

      # start with np.zeros(|V|), add seen
tokens
```



Step 3: Training

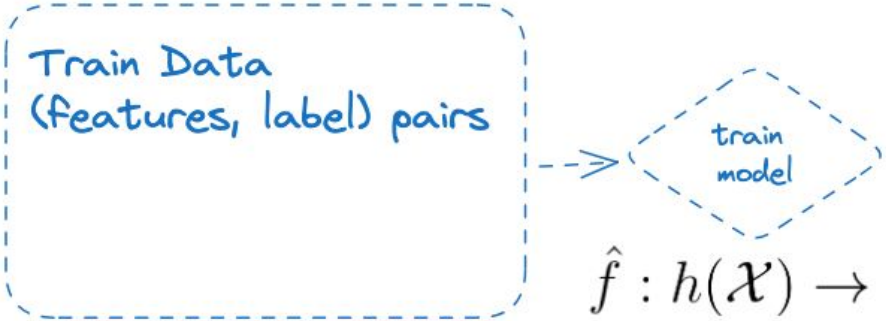
Class MyNaiveBayes:

def __init__(..):

def fit(X: npt.NDArray, y) ->:

X: 2D array (# sentences, |V|)

Y: 1D array (# sentences)



$$\hat{f} : h(\mathcal{X}) \rightarrow \mathcal{Y}$$

$$(h(\mathcal{X}_{train}) \times \mathcal{Y}_{train})$$

What do we need to compute?

Step 3: Training

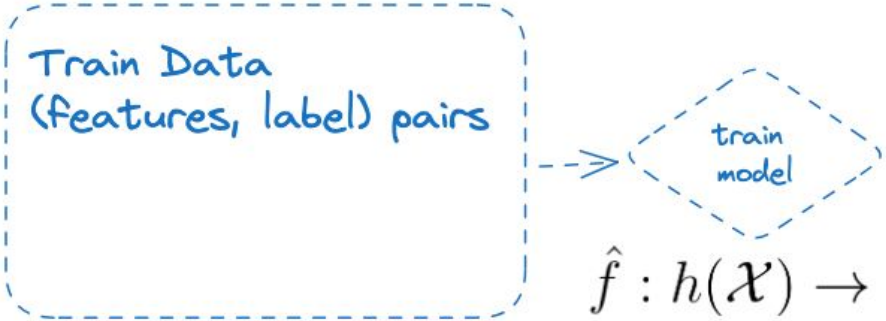
Class MyNaiveBayes:

def __init__(..):

def fit(X: npt.NDArray, y) ->:

X: 2D array (# sentences, |V|)

Y: 1D array (# sentences)



$$\hat{f} : h(\mathcal{X}) \rightarrow \mathcal{Y}$$

$$(h(\mathcal{X}_{train}) \times \mathcal{Y}_{train})$$

What do we need to compute?

priors and **likelihoods**

$$p(y|x_1 \ldots x_n) = \frac{\Pi_{i=1}^{n} p(x_i|y)p(y)}{p(x_1 \ldots x_n)} = \frac{\Pi_{i=1}^{n} p(x_i|y)p(y)}{\sum_{y' \in \mathcal{Y}} \Pi_{i=1}^{n} p(x_i|y')p(y')}$$

Estimating $p(x_i \mid y)$ and $p(y)$

	Max. Likelihood Estimate (MLE)	**Add-1 Smoothing (MAP estimate)**
$p(x_i \mid y = +)$	Word-count(word) in positive docs / total word-count in positive docs $$\frac{count(x = x_i \wedge y = y_i)}{\sum_{w \in \mathcal{V}} count(x = w \wedge y = y_i)}$$	
$p(y = +)$	Raw frequency of class (num. + docs / num. total docs): $$\frac{count(y = +)}{\sum_{y_i \in \mathcal{Y}} count(y = y_i)}$$	

Estimating $p(x_i \mid y)$ and $p(y)$

What **must** go in denominator for this prob. fact to still hold?

$$1 = \sum_{x_i \in \mathcal{V}} p(x = x_i | y = +)$$

	Max.	**Add-1 Smoothing (MAP estimate)**
$p(x_i \mid y = +)$	Word total $\frac{count(x = x_i \wedge y = y_i)}{\sum_{w \in \mathcal{V}} count(x = w \wedge y = y_i)}$	Assume we've seen every word nce-per-class $\frac{count(x = x_i \wedge y = y_i) + \mathbf{1}}{\sum_{w \in \mathcal{V}} count(x = w \wedge y = y_i) + ??}$
$p(y = +)$	Raw frequency of class (num. + docs / num. total docs): $\frac{count(y = +)}{\sum_{y_i \in \mathcal{Y}} count(y = y_i)}$	

Estimating p(x$_i$ | y) and p(y)

What **must** go in denominator for this prob. fact to still hold?

$$1 = \sum_{x_i \in \mathcal{V}} p(x = x_i | y = +)$$

	Max.	**Add-1 Smoothing (MAP estimate)**
p(x$_i$ | y = +)	Word total $\frac{count(x = x_i \wedge y = y_i)}{\sum_{w \in \mathcal{V}} count(x = w \wedge y = y_i)}$	Assume we've seen every word nce-per-class $\frac{count(x = x_i \wedge y = y_i) + \mathbf{1}}{\sum_{w \in \mathcal{V}} count(x = w \wedge y = y_i) + ??}$
p(y		

$$\hat{p}(x_i | y = y_i) = \frac{count(x = x_i \wedge y = y_i) + \mathbf{1}}{\sum_{w \in \mathcal{V}} [count(x = w \wedge y = y_i) + 1]} = \frac{count(x = x_i \wedge y = y_i) + \mathbf{1}}{\sum_{w \in \mathcal{V}} [count(x = w \wedge y = y_i)] + |\boldsymbol{V}|}$$

Step 4: Predict & Evaluate

Class MyNaiveBayes:

def __init__(..):

def fit(X: npt.NDArray, y) ->:

X: 2D array (# sentences, |V|)

Y: 1D array (# sentences)

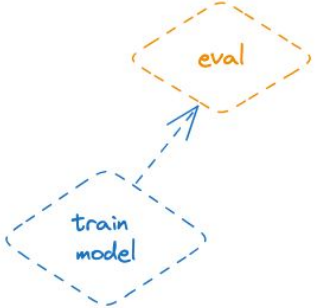
def predict(X: npt.NDArray) -> npt.NDArray:

X: 2D array (# sentences, |V|)

return: y_hat 1D array (# sentences)

$$(h(\mathcal{X}_{test}) \times \mathcal{Y}_{test})$$

$$Acc = \frac{1}{N} \sum_{(x,y) \in X_{test} \times Y_{test}} \mathbb{I}[(\hat{f}(h(x)) == y)]$$



$$\hat{f} : h(\mathcal{X}) \rightarrow \mathcal{Y}$$

$$\hat{y} = \underset{y}{argmax}\, p(y|x) = \underset{y}{argmax} \frac{\prod_{i=1}^{n} p(x_i|y)p(y)}{\sum_{y'} p(x_i|y')p(y')}$$

Step 4: Predict & Evaluate

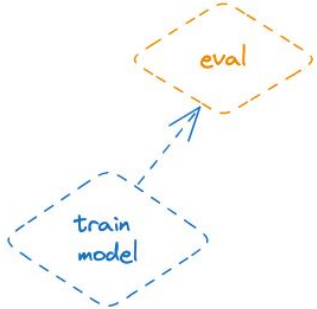
Class MyNaiveBayes:

```
def __init__(..):

def fit(X: npt.NDArray, y) ->:

    X: 2D array (# sentences, |V|)

    Y: 1D array (# sentences)

def predict(X: npt.NDArray) -> npt.NDArray:

    X: 2D array (# sentences, |V|)

    return: y_hat 1D array (# sentences)
```

$$(h(\mathcal{X}_{test}) \times \mathcal{Y}_{test})$$

Test Data
(features, label) pairs

$$Acc = \frac{1}{N} \sum_{(x,y) \in X_{test} \times Y_{test}} \mathbb{I}[(\hat{f}(h(x)) == y)]$$



$$\hat{f} : h(\mathcal{X}) \rightarrow \mathcal{Y}$$

$$\hat{y} = \underset{y}{argmax}\, p(y|x) = \underset{y}{argmax} \prod_{i=1}^{n} p(x_i|y)p(y)$$

Practice Not Graded 4

Build a binary classifier using Naive Bayes using this dataset. https://archive.ics.uci.edu/dataset/53/iris

Test the accuracy on 80-20 split.

Example Implementation

Practice not graded 6

In the general case, imagine that we have d binary features, and we want to count the number of features with value 1. How many leaf nodes would a decision tree need to represent this function?

Following are the target values predicted by a decision tree in a training dataset which we used to find whether a person have passed in interview or not.

[T, T, T, F, F, T, T, T]

What is the entropy H(pass)?

Some entropy/information theory asides

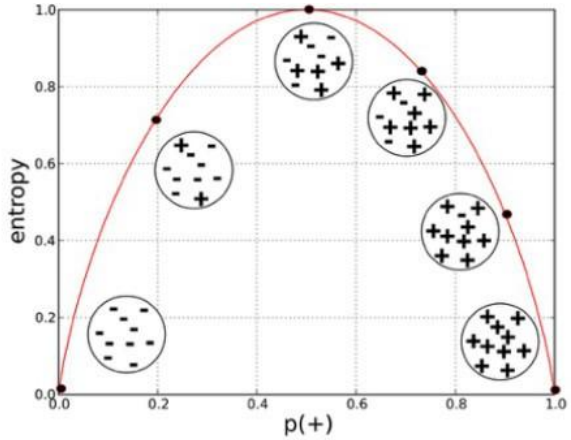
Information Gain is entropy reduced through an observation

1 ‘bit’ of entropy is enough information to distinguish two equiprobable alternatives

- If I use a time-machine to tell you the next flip of a fair coin is heads:
  - H([0.5, 0.5]) - H([1.0, 0]) = 1 - 0 → **1 bit of information gain**
- If I use a time-machine to tell you wednesday’s powerball numbers:
  - $-\sum_{i=0}^{292,201,338} \frac{1}{292,201,338} log(\frac{1}{292,201,338})$ - 0 → **~28.12 bits of information gain**
- The best decisions in a decision tree maximize *expected* information gain

Entropy: A way to measure impurity

- Entropy $H(X)$ of a random variable X:



Provost, Foster; Fawcett, Tom. Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking

$$H(X) = -\sum_{i=1}^{n} P(X=i)\log_2 P(X=i)$$

Entropy does not depend on X! Only P(X)

From entropy to information gain

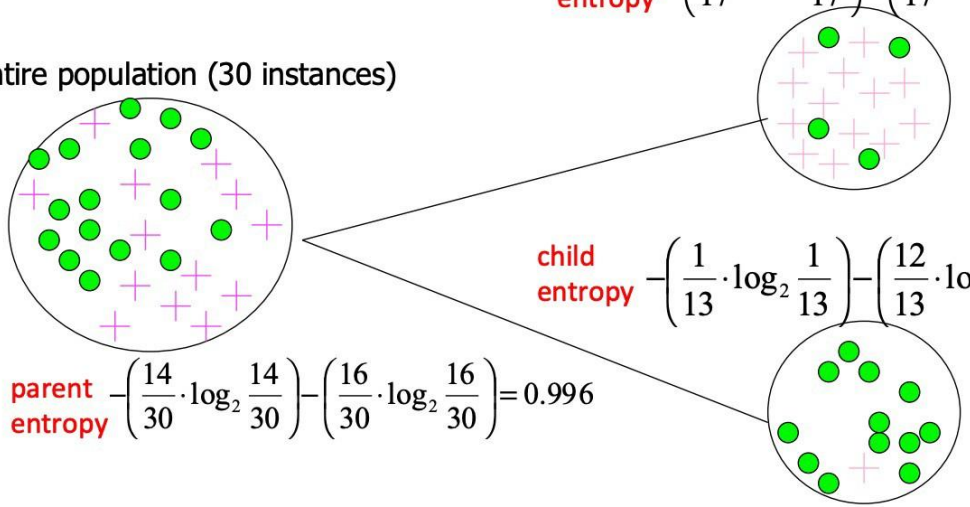
- How much more organized the labels become once we apply the decision stump?
- We calculate the entropy for each of the decision stump's leaves, and take the average of those leaf entropy values (weighted by the number of samples in each leaf)
- The information gain is then equal to the original entropy minus this new, reduced entropy.

Calculating Information

Information Gain = entropy(parent) – [average entropy(children)]

child entropy $-\left(\frac{13}{17}\cdot\log_2\frac{13}{17}\right)-\left(\frac{4}{17}\cdot\log_2\frac{4}{17}\right)=0.787$

Entire population (30 instances)



child entropy $-\left(\frac{1}{13}\cdot\log_2\frac{1}{13}\right)-\left(\frac{12}{13}\cdot\log_2\frac{12}{13}\right)=0.391$

parent entropy $-\left(\frac{14}{30}\cdot\log_2\frac{14}{30}\right)-\left(\frac{16}{30}\cdot\log_2\frac{16}{30}\right)=0.996$

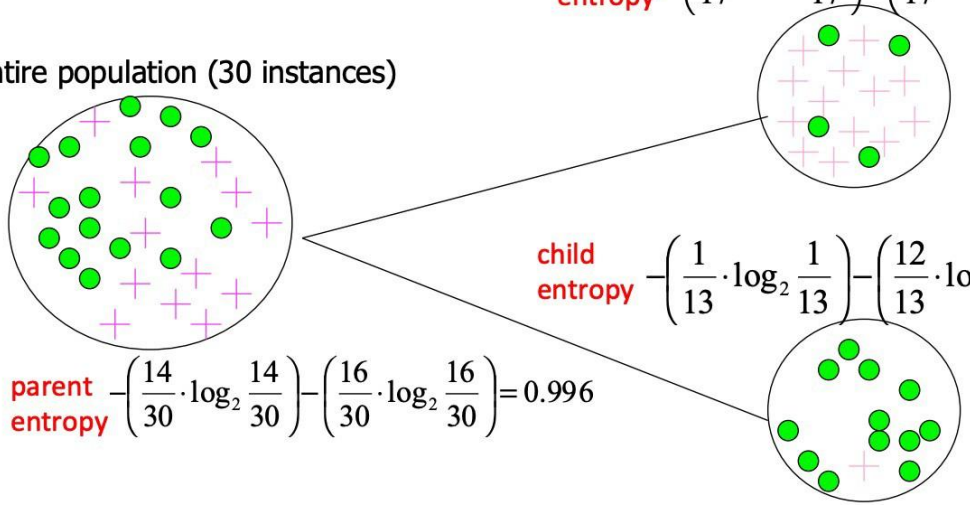
Based on slide by Pedro Domingos,

Calculating Information

Information Gain = entropy(parent) – [average entropy(children)]

child entropy $-\left(\frac{13}{17}\cdot\log_2\frac{13}{17}\right)-\left(\frac{4}{17}\cdot\log_2\frac{4}{17}\right)=0.787$

Entire population (30 instances)



child entropy $-\left(\frac{1}{13}\cdot\log_2\frac{1}{13}\right)-\left(\frac{12}{13}\cdot\log_2\frac{12}{13}\right)=0.391$

parent entropy $-\left(\frac{14}{30}\cdot\log_2\frac{14}{30}\right)-\left(\frac{16}{30}\cdot\log_2\frac{16}{30}\right)=0.996$

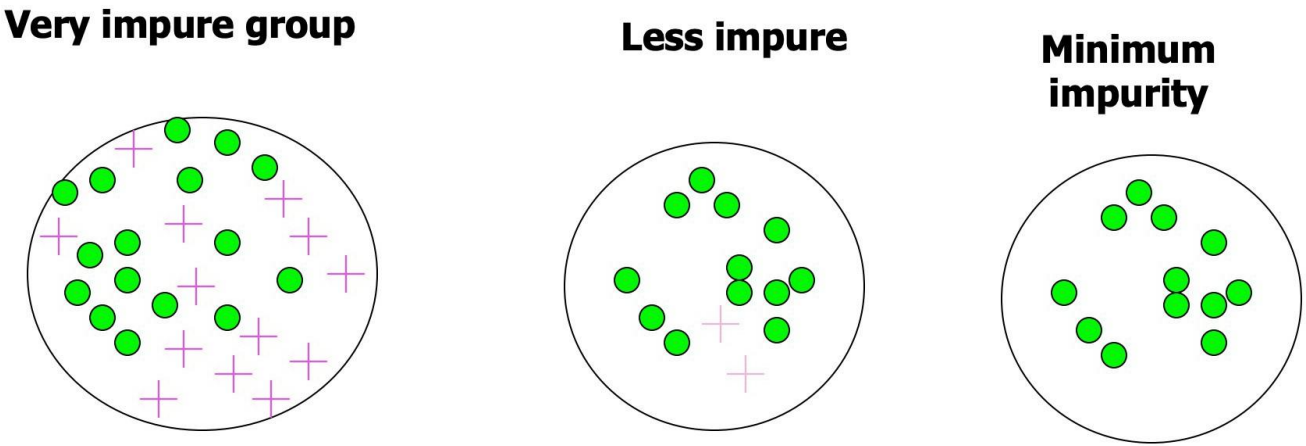
(Weighted) Average Entropy of Children $=\left\{\left(\frac{17}{30}\cdot 0.787\right)+\left(\frac{13}{30}\cdot 0.391\right)=0.615\right\}$

This is an expectation over H(Y|f={left,right})

Information Gain= 0.996 - 0.615 = 0.38

Based on slide by Pedro Domingos,

Impurity and Entropy



High Entropy

Medium Entropy

Low Entropy

Open Lab / Questions

More Information Theory/Entropy Notes

Information Theory: a Tutorial Introduction: I think sections 2-4, + 15 make most sense for ML, the remaining are somewhat more focused on communication channels